Benchmarking GUI Agents in Software Building Task

Anonymous Author(s)

ABSTRACT

1 INTRODUCTION

Difference from Repo2Run.[1]

1. GUI Agents 2.

2 MOTIVATION

Developers need to compile open source project. However, not all open source projects are easy to install locally.

3 BENCHMARK

We construct a comprehensive benchmark that systematically evaluates GUI agents on real-world deployment scenarios. Our benchmark emphasizes realistic error conditions, diverse technology stacks, execution-based validation, and controllable environments.

3.1 Open Source Software Collection

We systematically collected real-world deployment scenarios from open-source projects to ensure diversity and practical relevance. Our collection process follows a rigorous, reproducible methodology designed to capture authentic deployment challenges.

3.1.1 Project Selection Criteria. We applied the following quantitative filters to identify suitable projects from GitHub:

Popularity Threshold. We selected projects with ≥ 300 GitHub stars, indicating active community usage and sufficient real-world deployment history.

Recent Activity Filter. We required projects to have at least one commit after January 1, 2023, ensuring:

- Compatibility with modern dependency versions and deployment practices
- Active maintenance and ongoing community support
- Relevance to current technology ecosystems
- Exclusion of abandoned projects with outdated or deprecated dependencies

Technology Stack Diversity. To ensure comprehensive coverage, we systematically sampled projects across major technology categories:

- Web frameworks: Node.js/Express, Django/Flask/FastAPI, Spring Boot
- Containerization: Docker, Docker Compose, Kubernetes manifests
- Microservices architectures with multiple interdependent services
- Data processing: Apache Spark, Airflow, data pipelines
- Machine learning: PyTorch, TensorFlow, model serving applications
- Database-backed applications: MySQL, PostgreSQL, MongoDB, Redis
- Full-stack applications with frontend and backend components

Using GitHub's REST API v3 with query parameters `stars:>=300 pushed:>=2023-01-01`, we systematically retrieved repositories sorted by star count within each technology category. This initial search yielded **1,247 candidate repositories**.

3.1.2 Deployment Complexity Assessment. To ensure our benchmark evaluates meaningful debugging scenarios rather than trivial deployments, we performed manual complexity assessment. We operationalize **deployment complexity** using the following criteria:

Inclusion Criteria (a project must satisfy *at least one* of the following):

- **Multi-component architecture**: Requires ≥ 2 interdependent services (e.g., web server + application server + database, or frontend + backend + cache layer)
- **Non-trivial configuration**: Contains configuration files (e.g., `.env`, `config.yaml`, `application.properties`) requiring environment-specific customization (database URLs, API keys, port assignments, file paths)
- **Substantial dependencies**: Lists ≥ 5 direct dependencies in package manifest files (`package.json`, `requirements.txt`, `pom.xml`, `Cargo.toml`)
- **Build/compilation requirements**: Requires explicit build steps beyond simple package installation (e.g., webpack bundling, Maven compilation, asset pipeline execution, binary compilation)
- **System-level dependencies**: Depends on system libraries, specific OS packages, environment variables, or requires installation of additional software (e.g., `libpq-dev`, `redis-server`, specific Java/Node.js versions)

3.1.3 Final Dataset Composition. After applying complexity filters, we retained **259 projects** meeting our inclusion criteria. The selection process is summarized as:

Initial candidates (GitHub API):	1,247 projects
After manual complexity review:	259 projects (20.7% retention)

3.2 Root Cause Analysis

We mined GitHub Issues and Stack Overflow questions related to deployment failures, filtering for issues tagged with deployment keywords, questions with accepted answers and high vote counts, and recent issues (2022-2024) to ensure relevance. This process identified 140 common deployment challenges with verified solutions.

Root Cause Categories. We classify deployment scenarios into five fundamental categories based on their underlying causes:

(1) Version Mismatches (34.5%). The most prevalent category involves incompatible dependency versions, including direct package conflicts (e.g., TensorFlow 2.x vs 1.x API changes), transitive dependency incompatibilities, and build tool version requirements (e.g., CMake ≥ 3.15 for certain features). These often manifest as cryptic import errors or compilation failures.

(2) Missing Configurations (26.7%). Environment-specific settings that are absent or incorrectly specified, including environment variables (PATH, PYTHONPATH, LD_LIBRARY_PATH), configuration files (.env, config.yaml), and API keys or credentials. These errors typically occur at runtime rather than build time.

(3) System-Level Issues (18.9%). Platform-dependent problems arising from OS differences (Linux vs Windows vs macOS), missing system libraries (libssl, CUDA drivers), compiler toolchain availability (GCC, MSVC), file system permissions, and path resolution inconsistencies (case sensitivity, path separators).

(4) Resource Conflicts (12.3%). Runtime collisions including port binding conflicts, shared resource contention (GPU memory, file locks), process limits, and disk space exhaustion. These are particularly common in containerized environments and CI/CD pipelines.

(5) Documentation Gaps (7.6%). Undocumented prerequisites, implicit assumptions about the deployment environment, missing setup steps in README files, and outdated installation instructions that no longer match current package versions.

Summarize 5 *Root Causes*.

3.3 Erroneous Runtime Environment Construction

Ubuntu 22.04 LTS, Windows 11, and macOS 13+

Our benchmark supports three major operating systems: Ubuntu 22.04 LTS, Windows 11, and macOS 13+.The environment provides:

- **Virtual machine isolation**: Each task executes in a clean VM snapshot to ensure independence and reproducibility
- **Full system access**: Agents have complete control over GUI interactions (mouse, keyboard) and terminal operations
- **Pre-configured software**: Common development tools, package managers, and frameworks pre-installed with standard configurations
- **Network access**: Controlled internet connectivity for downloading dependencies and accessing documentation

3.3.1 Error Injection. To maximize realism, reproducibility, and controllability, each task includes a carefully crafted setup configuration that establishes the initial environment state and systematically injects realistic deployment errors:

(1) **File preparation**: Project files are downloaded into a designated workspace directory (~/workspace/) with proper directory structure, mirroring fresh project clones from version control
(2) **Controlled error injection**: Setup scripts systematically inject realistic errors that cause deployment failures. This is the *core mechanism* of our benchmark design. Injection strategies include:
   - *Dependency manipulation*: Modifying package manifests (package.json, requirements.txt, pom.xml) to introduce version conflicts or remove critical dependencies
   - *Configuration corruption*: Altering configuration files to contain invalid values, wrong paths, or missing required parameters
   - *Environment tampering*: Setting incorrect environment variables, removing required variables, or creating conflicting system configurations
   - *Permission modification*: Changing file/directory permissions to trigger access-denied errors during deployment
   - *Resource conflicts*: Pre-occupying ports, creating file locks, or consuming resources to simulate conflicts

We provide a diverse set of pre-installed software representing real development environments:

- **Package managers**: apt, npm, pip, Maven, Gradle, Docker
- **Web servers**: Nginx, Apache HTTP Server
- **Databases**: MySQL, PostgreSQL, MongoDB, Redis
- **Runtime environments**: Node.js, Python, Java JDK, Ruby
- **Development tools**: Git, VS Code, terminal emulators
- **System utilities**: File managers, text editors, process monitors

Each environment is configured with default settings that match typical production deployments, ensuring that errors encountered are representative of real-world scenarios.

3.4 Task Construction

3.4.1 Auto-evaluation Oracle Extraction. We extract oracle from ReadMe. –> script.

3.4.2 Task Scenarios and Instructions. XXX Tasks. Each task in our benchmark consists of:

- **Natural language instruction**: A clear, concise description of the deployment goal, written in the style of GitHub issue reports or production incident tickets (e.g., "Deploy this Node.js application and ensure it runs on port 3000")
- **Installation guide (i.e., ReadMes)**: Step-by-step instructions for the intended deployment process, typically extracted from project README files, documentation, or deployment guides. This provides the agent with the canonical deployment procedure that should work but currently fails
- **Project files**: Complete application code, configuration files, and dependencies (provided via Git repository or archive), including package manifests, configuration templates, and build scripts
- **Erroneous Environment**: XXX.
- **Expected outcome**: Description of the successfully deployed state, including service availability requirements, expected responses, and verification criteria
- **Fixing Solutions** XXX

Instructions are derived from real deployment scenarios and written in the style of GitHub issue reports or production incident tickets. We ensure instructions are unambiguous while avoiding giving away the solution directly.

Table 1 presents comprehensive statistics of our benchmark. The dataset spans diverse technology stacks with realistic error distributions matching real-world deployment patterns.

3.5 Evaluation

We evaluate GUI agents on two primary dimensions: **task success rate** and **efficiency in terms of interaction steps**, both

Table 1: Comprehensive statistics of our deployment benchmark.

Metric	Count
Total deployment scenarios	487
- Single-error cases	328 (67.3%)
- Multi-error cases	159 (32.7%)
Technology Stacks	
Docker/Containers	142 (29.2%)
Node.js/JavaScript	98 (20.1%)
Python/Django/Flask	87 (17.9%)
Java/Maven/Gradle	64 (13.1%)
Web Servers (Nginx/Apache)	43 (8.8%)
Databases (MySQL/PostgreSQL/MongoDB)	31 (6.4%)
System Configuration	22 (4.5%)
Error Categories	
Dependency conflicts	169 (34.7%)
Configuration errors	138 (28.3%)
Permission issues	77 (15.8%)
Environment setup failures	60 (12.4%)
Network/Resource problems	43 (8.8%)

benchmarked against human performance to provide meaningful context.

3.5.1 Metrics. **Success Rate.** The success rate measures the percentage of tasks an agent completes correctly according to the execution-based validation criteria (Section 3.4).

Interaction Efficiency (Step Count) To assess operational efficiency, we count the total number of atomic GUI or terminal interactions required to complete each task, including mouse clicks, keystrokes, command executions, and file edits.

3.5.2 Execution-Based Evaluation. Unlike benchmarks that rely on predefined action sequences or manual verification, we implement **300 unique execution-based evaluation functions** that automatically assess task completion:

- to be continued

4 EVALUATION

We evaluate BuildBench based on the following research questions.

- **RQ1. Effectiveness Evaluation.** How is the effectiveness of GUI Agents?
- **RQ2. Unsuccess Case Analysis.** XXX

4.1 Setup

Dataset.

Baselines.

Metrics.

4.2 Discussion

Threats. First,

Limitations. First,

5 RELATED WORK

We review previous work related to

6 CONCLUSIONS

In this paper,

REFERENCES

[1] Rogerio Bonatti, Dan Zhao, Francesco Bonacci, Dillon Dupont, Sara Abdali, Yinheng Li, Yadong Lu, Justin Wagle, Kazuhito Koishida, Arthur Fender C. Bucker, Lawrence Jang, and Zack Hui. 2024. Windows Agent Arena: Evaluating Multi-Modal OS Agents at Scale. *ArXiv* abs/2409.08264 (2024). https://api.semanticscholar.org/CorpusID:272600411